

TAGRO OS: Cloudflare Storage & Uploads Reference

Data Topology, Sync Pipelines, and Edge Storage Architectures (Updated June 2026)

This document provides a clear architectural blueprint of the data flow, storage endpoints, and sync pipelines across the TAGRO OS ecosystem. It serves as a technical blueprint for the layout of our databases, storage buckets, local parsers, and client-side memory structures.

1. The Core Philosophy of Data in TAGRO OS

Following the **TAGRO OS Philosophy of Cleanliness and Honesty**, the database architecture is designed to enforce clarity:

- **The software exists to support people.** That means data structures must prioritize performance and context preservation over complex relational logic.
- **Attention is the most valuable resource.** The client-side database is designed to run *offline-first* using local caches (`localStorage`). Mobile tablets query local memory with sub-millisecond responses, completely immune to cellular dropouts or slow workshop Wi-Fi.

2. Local-to-Cloud Pipeline (The Local Laptop Parser)

The python uploader utility (`tagro_catalog_upload.py`) acts as the compiler. It processes heavy, nested raw spreadsheets and PDF catalogs locally on your machine and uploads structured, lightweight payloads to the Cloudflare edge.

```
[Local Laptop]
├─ Stihl MS_Parts 2024.xlsx → (Python Parser) → [/admin/kv-set] → [Cloudflare KV]
├─ MS 250 parts catalog.pdf → (PDF Splitting) → [/admin/r2-put] → [Cloudflare R2]
└─ Prices_STIHL June 2026.xlsx
```

The Ingestion Workflow

1. **Spreadsheet Ingest:** The Python script reads structural sheets, normalizes part numbers, formats coordinates for hotspots, and packs them into JSON arrays.
2. **PDF Ingest:** It slices each page of the manual PDF, rasterizes them as web-friendly PNG images, and extracts the raw page-level index.
3. **Edge Handshake:** It signs requests using the `X-TAGRO-Key` (matching the Worker's `env.OWNER_TOKEN`) and transfers the packed payloads using chunked streams to bypass buffer limits.

3. Cloudflare Storage Architecture (The Edge Layer)

Cloudflare acts as the distributed backend, splitting data into three specialized storage classes based on access speed and data type.

A. Cloudflare KV (Key-Value Store)

KV is designed for read-heavy, low-latency configuration and structural data. The following namespace schema holds our entire operational catalog:

Key Pattern	Data Structure / Value Type	Description / Usage
<code>parts:_index</code>	JSON Object	Global index containing all loaded models, categories, total parts, and sheet section lists.
<code>parts:{MODEL_KEY}</code>	JSON Array	High-performance catalog of all sections and parts parsed from master spreadsheets (e.g. <code>parts:MS250</code>).
<code>parts-price:{MODEL_KEY}</code>	JSON Object	Local/Individual model pricing spreadsheet matrix containing custom dealer margins and MRP allocations.
<code>parts-price:master_lookup</code>	JSON Object	Master Ingest Matrix: Normalized exact part keys map directly to June 2026 MRPs, dealer costs, GST percentages, and HSN codes. Used globally by the AI Assistant.
<code>manual-text:{MODEL_KEY}</code>	JSON Object	Extracted raw text blocks, heading indexes, and page markers from PDF plumber operations. Used by Claude to ground manual-specific queries.
<code>staff:all</code>	JSON Array	Active directory of authorized employees, branch designations, roles, and registered phone numbers.
<code>models:all</code>	JSON Array	Global directory of all supported STIHL machines and their active branch inventory distribution list.
<code>otp:{USER}_{BRANCH}</code>	JSON Object	Single-use 6-digit numeric login codes with 10-minute expirations and self-destructing attempt locks.
<code>device:{USER}_{BRANCH}</code>	JSON Object	Cryptographic hardware device bindings, associating tablet device IDs with authorized staff logins.

B. Cloudflare R2 (Object Storage)

R2 acts as our lightning-fast, zero-egress asset bucket, storing high-resolution page diagrams.

- **Bucket Name:** `tagro-manuals` (represented as `env.R2`)
- **Path Structure:** `stihl/{MODEL_KEY}/pages/p{page_no}.png`
- **Access Control:** Served securely through the Cloudflare Worker proxy, or via a mapped custom domain (e.g. `$https://manuals.tagro.in$`).

C. Cloudflare D1 (Relational Database)

The specialized `tagro-service-ingest` worker binds directly to a Cloudflare D1 SQLite instance (`env.DB`), handling transactional logs, automated incoming branch backups, and staging tables.

- `service_jobs` : Holds the primary master service records promoted to production.
- `staging_jobs` : Holds incoming raw rows uploaded from Dropbox branch backups waiting to undergo Claude validation for anomalies (hard errors, missing serial numbers).

4. Client-Side Lifecycle (The Tablet Layer)

To protect your staff's attention from network lag, `app.js` uses a **cache-first lifecycle**:

```
[Page Load]
├── 1. Read Cached Data from localStorage (Instant Render)
└── 2. Silent Background Version Check:
    Client requests: GET /config/all?v_staff={v}&v_parts={v}
    Worker response:
        ├── If versions match: HTTP 200 (No changes, skip download)
        └── If version outdated: Worker sends JSON updates -> Saves to local
```

Local Caching Map

LocalStorage Key	Local Memory Pointer	Native Memory Array Description
<code>tagro_kv_staff</code>	<code>kvStaff()</code>	Cached list of branch employees used for local authorization overrides.
<code>tagro_kv_models</code>	<code>kvModels()</code>	Cached array of supported machines.
<code>tagro_parts_data</code>	<code>partsData()</code>	Client-Side Master Price List: Holds the active copy of the 2022 master parts table, merged locally with June 2026 specs.
<code>tagro_session</code>	<code>session()</code>	Current authenticated user session details, login timestamp, and role parameters. Expires in 12 hours.
<code>tagro_jobs</code>	<code>jobs()</code>	Local copy of active branch work orders, merged with incoming Dropbox instances.

5. The Normalization Engine (`norm` and `clean_part_no`)

At the core of matching local inputs with June 2026 master prices is the **Part Normalization Engine**. This algorithm runs identically in your local Python script, on your Cloudflare Workers, and in your client-side search scripts:

```
function norm(s, isQuery) {
  if (!s) return "";
  let str = String(s).trim();
  if (str.endsWith('.0')) {
    str = str.slice(0, -2);
  }
  // Strip all non-alphanumeric characters (spaces, dashes, slashes, dots)
  str = str.replace(/[^A-Za-z0-9]/g, '');

  // Pad numeric strings under 11 digits back to standard STIHL format
  if (/^\d+$/.test(str) && str.length < 11) {
    if (!isQuery || str.length >= 7) {
      str = str.padStart(11, '0');
    }
  }
}
```

```

    }
  }
  return str.toLowerCase();
}

```

Match Scenarios

Raw Excel Input	User Typed Query	Normalized Key	Status
1123-640-2000	1123 640 2000	11236402000	Exact Match
1123 640 2000	11236402000	11236402000	Exact Match
4007000 (<i>dropped zeros</i>)	0000-400-7000	00004007000	Exact Match (Smart Padded)
3005 000 4813.0	3005-000-4813	30050004813	Exact Match (Float Stripped)

6. Maintenance & Updates Checklist

When you need to perform pricing or structural maintenance on TAGRO OS:

1. **Spreadsheet Updates:** To update spare parts lists, simply add/modify the rows in `Prices_STIHL June 2026.xlsx` in your local folder and run `tagro_catalog_upload.py`. The uploader handles the formatting, parsing, and pushing automatically.
2. **Code Modifications:** Any adjustments to the visual presentation (`catalog.html`), local matching routines (`parts-search.js`), or menu navigation (`app.js`) must be committed and pushed directly to your **GitHub repository** to go live.
3. **Database Maintenance:** Backups of jobs and staging records are handled automatically by D1, with structured JSON archives stored inside your branch folder structures on **Dropbox** for absolute redundancy.